RNGDet: Road Network Graph Detection by Transformer in Aerial Images

Zhenhua Xu, *Graduate Student Member, IEEE*, Yuxuan Liu, *Graduate Student Member, IEEE*, Lu Gan, *Student Member, IEEE*, Yuxiang Sun, *Member, IEEE*, Xinyu Wu, *Senior Member, IEEE*, Ming Liu, *Senior Member, IEEE*, and Lujia Wang, *Member, IEEE*

***Abstract*— Road network graphs provide critical information for autonomous-vehicle applications, such as drivable areas that can be used for motion planning algorithms. To find road network graphs, manual annotation is usually inefficient and labor-intensive. Automatically detecting road network graphs could alleviate this issue, but existing works still have some limitations. For example, segmentation-based approaches could not ensure satisfactory topology correctness, and graph-based approaches could not present precise enough detection results. To provide a solution to these problems, we propose a novel approach based on transformer and imitation learning in this article. In view of that high-resolution aerial images could be easily accessed all over the world nowadays, we make use of aerial images in our approach. Taken as input an aerial image, our approach iteratively generates road network graphs vertex-by-vertex. Our approach can handle complicated intersection points with various numbers of incident road segments. We evaluate our approach on a publicly available dataset. The superiority of our approach is demonstrated through comparative experiments. Our work is accompanied by a demonstration video which is available at https://tonyxuqaq.github.io/projects/RNGDet/.**

***Index Terms*— Aerial images, autonomous driving, imitation learning, remote sensing, road network graph detection, transformer.**

Manuscript received 25 February 2022; revised 2 May 2022 and 2 June 2022; accepted 2 June 2022. Date of publication 29 June 2022; date of current version 12 July 2022. This work was supported in part by the Zhongshan Science and Technology Bureau Fund under Project 2020AG002 and in part by Foshan-The Hong Kong University of Science and Technology (HKUST) under Project FSUST20-SHCIRI06C. The work of Ming Liu was supported in part by the Guangdong Basic and Applied Basic Research Foundation under Project 2020A0505090008. *(Corresponding author: Lujia Wang.)*

Zhenhua Xu is with the Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong (e-mail: zxubg@connect.ust.hk).

Yuxuan Liu and Lu Gan are with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong (e-mail: yliuhb@connect.ust.hk; lganaa@connect.ust.hk).

Yuxiang Sun is with the Department of Mechanical Engineering, The Hong Kong Polytechnic University, Hong Kong (e-mail: yx.sun@polyu.edu.hk).

Xinyu Wu is with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China (e-mail: xy.wu@siat.ac.cn).

Ming Liu is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Guangzhou, Guangdong 511400, China, also with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, and also with the HKUST Shenzhen-Hong Kong Collaborative Innovation Research Institute, Futian, Shenzhen 518040, China (e-mail: eelium@ust.hk).

Lujia Wang is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, and also with the Clear Water Bay Institute of Autonomous Driving, Shenzhen 518057, China (e-mail: eewang@ust.hk).

Digital Object Identifier 10.1109/TGRS.2022.3186993

I. Introduction

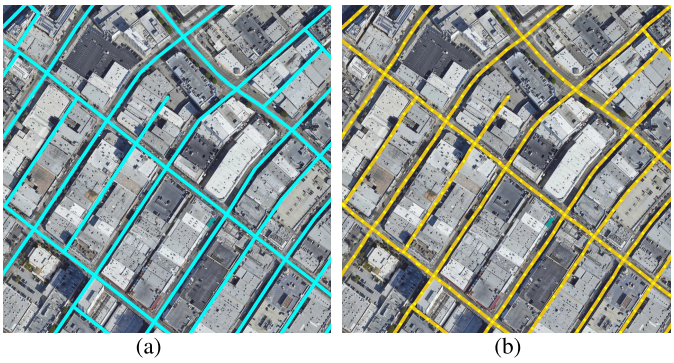
In RECENT years, road networks have attracted considerable attention in the field of autonomous driving. The graph of road networks can provide fundamental information for autonomous-vehicle applications. The graph of road networks is a kind of vectorized data representation, which consists of vertices and edges [1]. Each road segment could be seen as a graph edge, and the intersection points of road segments are vertices. Manually annotating the road network graph is time-consuming and labor-intensive, especially when road networks cover a large area (e.g., a whole city). Therefore, how to automatically detect road network graphs using automatic algorithms in large areas is of great interest to the research community.

To address this issue, past approaches to road network graph detection usually use aerial images obtained from unmanned aerial vehicles (UAVs) or satellites [2]. As aerial imaging technology evolves, high-resolution and high-quality aerial images can be easily accessed worldwide nowadays. Moreover, some aerial imaging datasets also provide extra channels besides red–green–blue (RGB), such as the infrared channel [3], making them more informative for detection purposes. So, this work also uses aerial images for road network graph detection.

Existing works on road network graph detection can be generally classified into two categories: segmentation-based approaches [2], [4]–[13] and graph-based approaches [14]–[18]. The segmentation-based approaches first predict the probabilistic segmentation map of the road network graph and then conduct a series of processing to obtain the graph structure of the road network, such as skeletonization and filtering. Most of the early works on road network graph detection in this field fall into this category. The segmentation-based approaches could present good results in the pixel-level evaluation (e.g., by F1 score) due to the use of existing powerful semantic segmentation networks, but they usually suffer from unsatisfactory topology correctness such as incorrect crossroad connectivity and false disconnection on the road. To address this issue, recent graph-based approaches resort to detecting the graph of road networks directly [14], [15], [17], [18]. They usually first predict candidate initial vertices, then, starting from each candidate's initial vertex, train a decision-making agent to predict adjacent vertices of the current vertex. In this way, road network graphs can be generated vertex-by-vertex in an iterative manner. Although these graph-based approaches could enhance the topology correctness, they are usually composed of two separate stages, making them hard to

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 1. Sample result of RNGDet. (a) Ground-truth road network graph (cyan lines). (b) Graph predicted by RNGDet (orange lines for edges and yellow points for vertices). We can see that RNGDet effectively detects the road network graph with high quality. For better visualization, the lines are drawn with a thicker width while it is actually one-pixel width. The figure is best viewed in color. See zoomed-in view for details.

be optimized in an end-to-end way. The separate stages might accumulate errors and hence degrade their effectiveness and efficiency.

To provide a solution to these issues, in this article, we propose a graph-based end-to-end approach named road network graph detection by a transformer (RNGDet). Similar to previous graph-based approaches, RNGDet starts from predicted candidate initial vertices to extract local visual features using a convolutional neural network (CNN) backbone and then sends the features to a transformer network inspired by the DETR structure [19]. Due to the use of deep vertex queries, RNGDet can directly predict any number of adjacent vertices of the current vertex at one time, so that it can handle any road networks, even those with complicated topology (e.g., road intersections of arbitrary numbers of road segments). Different from previous graph-based approaches, RNGDet can be optimized as a whole and trained end-to-end. We train RNGDet through imitation learning to enable it to take the most appropriate action under different circumstances. To generate the training data (i.e., an expert demonstration from the imitation learning perspective), we propose a sampling algorithm to supervise the agent to explore the whole road network. The proposed RNGDet is trained and evaluated on a publicly available dataset released by RoadTracer [14]. With this dataset, we compare RNGDet with state-of-the-art works based on multiple evaluation metric scores. An example of RNGDet is visualized in Fig. 1. The contributions of our work are summarized as follows.

1) We propose an end-to-end trainable approach named RNGDet based on transformer and imitation learning to automatically detect the road network graph.
2) We propose an algorithm to automatically generate training samples for RNGDet.
3) We evaluate RNGDet and compare it with state-of-the-art works on a publicly available dataset to demonstrate the superiority of RNGDet.

The remainder of this article is organized as follows. Section II introduces related works. Section III describes the structure and working pipeline of RNGDet. Section IV presents experimental results, discussions, and limitations. Conclusions and future work are drawn in the last section.

II. Related Works

A. Segmentation-Based Approaches

Segmentation-based road network graph detection approaches [2], [4]–[13], [20]–[22] mainly have two stages: 1) predict the segmentation map (i.e., probabilistic map of the road network) and 2) process the segmentation map and obtain graph structures by post-processing, such as skeletonization and heuristic-based algorithms [23], [24]. It is believed that Mnih and Hinton [2] proposed the first work that implemented neural networks to detect the road network in aerial images. They first split the large aerial image into small patches, then predicted the road network within each patch, and finally merged patches into the final predicted road network segmentation map. Most later segmentation-based works followed a similar pipeline, but with more powerful segmentation networks, such as U-Net [25], DeepLab V3+ [26], FPN [27], and HRNet [28]. Batra *et al.* [8] extended the aforementioned works by adding another refinement network to fix incorrect pixels in the predicted segmentation map, which effectively improved the final performance. Gedara Chaminda Bandara *et al.* [11] proposed a spatial and interaction space graph reasoning (SPIN) module, which performed reasoning over graphs constructed on spatial and interaction spaces projected from the feature maps. In this way, spatial and topology information are better utilized to improve road detection performance. Generative adversarial network (GAN) is also a popular tool for map generation tasks [20]–[22], but usually, these works output maps in raster format, such as styled map tiles. Thus, postprocessing is still needed to vectorize maps.

After obtaining the probabilistic map of a road network, postprocessing should be conducted to extract the road centerline, filter out outlier road segments and fix incorrect disconnections of the road network. To better extract the road-network graph, Bulatov *et al.* [24] proposed similarity criteria to fuse raw road segments into chains and designed a two-stage algorithm to optimize the obtained road chains. This work can handle curve roads and sharp circle arcs well. Wenzel *et al.* [23] further enhanced [24] to generate longer and more accurate road network chains by proposing an iterative greedy optimization procedure.

Since semantic segmentation only works on pixel-level predictions, the topology information is not effectively considered. Thus, the road network graph obtained by segmentation-based approaches tends to have poor topology correctness. Moreover, handcrafted or heuristic postprocessing algorithms cannot effectively correct errors in the obtained road network graph.

B. Graph-Based Approaches

Different from segmentation-based approaches, graph-based approaches for road network graph detection can directly output the graph structure. Most of the graph-based works detect the road network graph by iterative graph generation [14]–[18]. RoadTracer [14] is believed to be the first work that iteratively generates the road network graph. In this work, ground-truth initial vertices were used to initialize the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

iteration. From each initial vertex, RoadTracer predicted the direction of the adjacent vertices of the current vertex as a multiclass classification problem, then moved the agent in the predicted direction by a fixed length. RoadTracer presented much better topology performance than past segmentation-based approaches, but it failed to detect road intersections with high quality due to the fixed step length. Tan *et al.* [17] solved this problem by replacing the direction prediction with heatmap prediction, where heatmap demonstrated the probabilistic distribution of adjacent vertices of the current vertex. After obtaining the heatmap, the authors extracted local peaks as the predicted adjacent vertices. Although this approach had dynamic step length and presented superior performance to past works, it cannot be optimized in an end-to-end way due to the postprocessing of the heatmap, which degrades the final performance. In addition, this work cannot distinguish vertices that are close to each other.

Different from aforementioned iterative graph-based approaches, He *et al.* [16] proposed Sat2Graph to directly predict the road network graph. Taken as input an aerial image, Sat2Graph predicted a 19 dimension tensor. This high-dimensional tensor contained all the information of the road network graph so that the graph could be decoded from the feature tensor by algorithms proposed in Sat2Graph [16]. However, Sat2Graph had the isomorphic encoding issue analyzed in [16], which made it difficult to supervise during training. Moreover, it cannot distinguish road segments whose intersection angle is small.

C. Graph Detection of Objects Similar to Road Networks

There are some tasks that are similar to road network detection, such as the detection of road boundaries [29]–[31], road lane lines [32]–[35], road lane [36], [37], and road curbs [38], [39]. Even though these works do not work on road network detection, their tasks are similar to ours and some ideas or techniques are also inspiring to us. Xu *et al.* [38] first proposed to analyze the graph detection problem using imitation learning and designed a DAgger-based system for road curb detection following the DAgger algorithm [40]. Homayounfar *et al.* [33] proposed DagMapper to detect road lane line graph in the point cloud map on the highway. DagMapper can predict the direction of adjacent vertices of the current vertex, and whether the agent should create a new lane line branch when lane line intersections were encountered. Although these works could be inspiring for our task, they cannot handle the road network graph detection task since road networks have much more complicated topology (e.g., road split, road merge, and crossroads).

D. Transformer-Based Detection

In recent years, transformer [41] has been receiving more and more attention since its powerful parallelization capacity and great ability to handle sequential tasks. Considering these properties, Carion *et al.* [19] proposed detection by a transformer (DETR) for one-shot 2-D object detection, which is anchor-free and can be trained in an end-to-end way. After extracting image features by a CNN backbone, DETR sent the obtained features as well as multiple object queries to a transformer and then obtained the bounding box coordinates of objects. Each bounding box was encoded by a 4-D embedding. Therefore, taken as input an image, DETR can directly output the coordinates of object bounding boxes. Xu *et al.* [42] adapted DETR to line segment detection task and named the new model as line segment detection by transformer (LETR). In this article, the authors encoded each line segment as an embedding and predicted the encoding embedding by a DETR-based network. Similarly, Can *et al.* [43] modified the DETR network and pursued to detect lane centerlines. Each lane centerline was fit by a B-spline and each B-spline was encoded by an embedding. In this way, the authors could directly detect all lane centerlines at one time by predicting the embedding that encoded B-spline information. Our work RNGDet is also inspired by DETR, while the output embedding encodes the information of adjacent vertices of the current vertex.

E. Imitation Learning

Imitation learning aims to train a decision-making agent network to mimic an expert. The most commonly used imitation learning algorithm is behavior cloning [44]. In behavior cloning, an expert generates a set of demonstrations and the agent tries to learn the policy of the expert. This algorithm is efficient but it suffers from the drifting problem of imitation learning [45]. Ross *et al.* [40] proposed a meta-algorithm called dataset aggregation (DAgger) which could cover a much larger state space to relieve the drifting problem. However, the DAgger algorithm presents quite a low sampling efficiency. Based on DAgger, Xu *et al.* [30], [38] designed approaches to detect road elements by imitation learning. In this article, since behavior cloning can already achieve satisfactory performance, considering the low-efficiency performance of DAgger, we adopt behavior cloning in our experiments.

III. Proposed Approach

A. Approach Overview

This work aims to detect the road network graph from aerial images, and the road network graph can be used for real-world applications (e.g., autonomous vehicle navigation). Suppose the input is a large aerial image I_A, then the final output should be a graph $G = (V, E)$. $E = \{e_i\}$ is a set of graph edges and each edge e_i represents a road segment. $V = \{v_j\}$ is a set of graph vertices, and each vertex v_j is either one intersection point of some road segments or the endpoint of a broken road.

Based on the DETR structure, our proposed RNGDet detects the road network graph by iterations. Starting from a prepredicted candidate initial vertex in $C = \{c_k\}_{k=1}^{K}$, RNGDet iteratively generates the road network graph by controlling an agent exploring the road network. During iterations, the historical trajectory of the agent is recorded by the graph G. At each step, centering at the current position of the agent v_t, RNGDet crops an ROI $\in \mathbb{R}^{3\times L\times L}$ on I_A and rasterizes G within the ROI as $\mathcal{H} \in \mathbb{R}^{1\times L\times L}$. A CNN backbone network is utilized to extract the deep visual feature of the ROI as $\mathcal{F}_I$ which is sent to the segmentation heads to predict the road segment segmentation map $\mathcal{S}$ and the road intersection

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

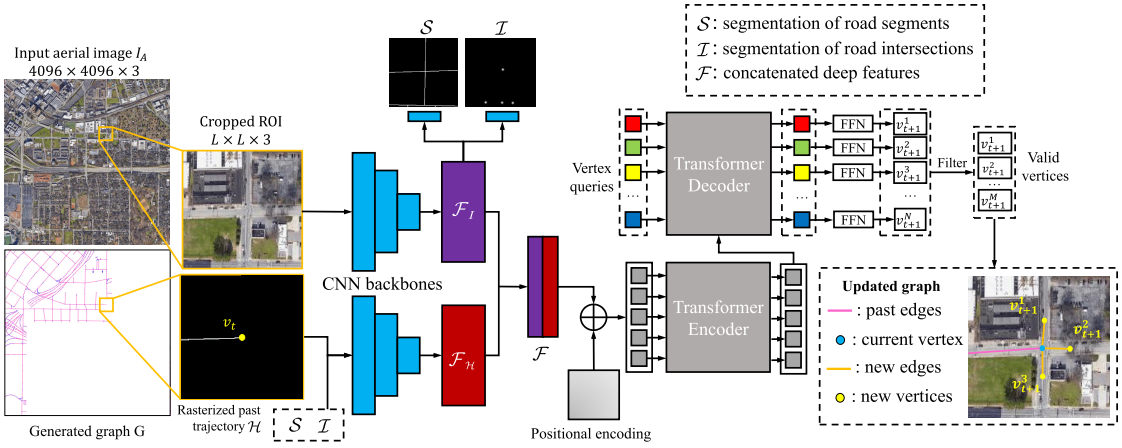


Fig. 2. System overview of RNGDet. The figure shows a single step in the graph detection process of RNGDet. In this example, RNGDet processes a road intersection. Suppose the agent is at v_t at the current time step (denoted by a yellow node in $\mathcal{H}$), we crop an ROI on the input aerial image I_A and rasterize the graph generated so far G within the ROI as $\mathcal{H}$. A CNN backbone network predicts the feature $\mathcal{F}_I$ of the ROI. Then, $\mathcal{F}_I$ is sent to the segmentation heads to predict $\mathcal{S}$ and $\mathcal{I}$. Taken as input $\mathcal{S}$, $\mathcal{I}$, and $\mathcal{H}$, another CNN network predicts the deep feature tensor $\mathcal{F}_\mathcal{H}$ and concatenates it with $\mathcal{F}_I$ as the final feature tensor $\mathcal{F}$. The transformer predicts the adjacent vertices $\mathcal{V} = \{v_{t+1}^i\}_{i=1}^M$ of v_t. In the updated graph of this example, there are three predicted adjacent vertices $\{v_{t+1}^i\}_{i=1}^3$ (yellow nodes), and they are connected with the current vertex v_t (blue node) by three new edges (orange lines). The graph G is generated in this way iteratively. This figure is best viewed in color. See zoomed-in view for details.

segmentation map $\mathcal{I}$. The candidate initial vertices in C can be obtained by finding local peaks of $\mathcal{I}$. After concatenating $\mathcal{H}$ with $\mathcal{S}$ and $\mathcal{I}$, another CNN backbone network is used to extract the deep feature $\mathcal{F}_\mathcal{H}$. Then, $\mathcal{F}_\mathcal{H}$ and $\mathcal{F}_I$ are fused as the input feature tensor of the transformer.

Taking N vertex queries $Q = \{q_i\}_{i=1}^N$ as input, the transformer decoder directly predicts N vertex embeddings encoding the valid probability and coordinates of vertices. Each vertex query is a learned embedding, which could be treated as a slot used by RNGDet to make the prediction of one vertex. Thus, the number of vertex queries N must be larger than the largest number of vertices needed to be predicted. After filtering out vertices with low probability, we obtain $M (M \leq N)$ valid vertices that are adjacent to v_t as a set $\mathcal{V} = \{v_{t+1}^i\}_{i=1}^M$. If $M = 0$, RNGDet pops a new candidate initial vertex from C and repeats the above process; if $M = 1$, the agent moves to the predicted coordinate and repeats the above process; if $M > 1$, the agent pushes all the predicted vertices into set C, pops one vertex from set C and then repeats the above process. When C is empty, RNGDet stops and outputs the final road network graph. The overall pseudocode of our system is shown in Algorithm 1. The system diagram of RNGDet is displayed in Fig. 2.

Algorithm 1 RNGDet

Input: An aerial image I_A
Output: The road network graph $G = (V, E)$

```
C ← find_initial_vertex(I_A)
Initialize G as an empty graph
while C not empty do
    t ← 0
    v_t ← C.pop()
    while true do
        F ← CNN(I_A, G, v_t)
        {v̂_{t+1}^i}_{i=1}^N ← Transformer(F|Q)
        V ← filter({v̂_{t+1}^i}_{i=1}^N)
        else if |V| = 0 then
            break
        else if |V| = 1 then
            v_t ← v̂_{t+1}^1
            t ← t + 1
            update G
        else if |V| > 1 then
            C ← C ∪ V
            update G
            break
    end
end
return G
end
```

B. CNN Backbone and Segmentation

The input of RNGDet is a large RGB aerial image $I_A \in \mathbb{R}^{3\times4096\times4096}$. Since RNGDet requires the history information for iterative graph generation, we maintain the graph G recording the past trajectory of the agent. Due to the size of input images, RNGDet crops an ROI$\in \mathbb{R}^{3\times L\times L}$ (L is 256 in our experiment) on I_A. A multilayer CNN is utilized to extract the deep feature of the ROI as $\mathcal{F}_I$, and the CNN backbone in this article is ResNet [46]. Based on the extracted deep feature $\mathcal{F}_I$, two feature paradigm network (FPN) [27] segmentation heads predict the segmentation map $\mathcal{S}$ and $\mathcal{I}$, where $\mathcal{S}$ demonstrates the distribution of road segments and $\mathcal{I}$ shows the distribution of road intersection points as well as endpoints of broken roads. Both segmentation tasks are binary segmentation.

To obtain the information of past trajectories, we crop G in the same way as cropping the ROI and rasterize the cropped G

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

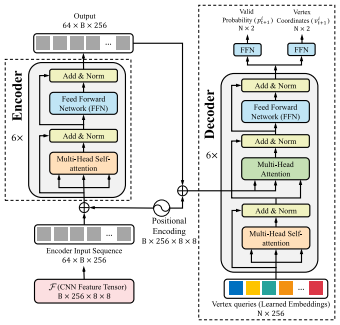


Fig. 3. Architecture of the transformer in RNGDet. In this figure, B represents batch size, and N represents the number of vertex queries. Taken as input the predicted CNN feature tensor $\mathcal{F}$, the transformer first splits $\mathcal{F}$ into a sequence of smaller tensors. In our experiment, the $B \times 256 \times 8 \times 8$-sized feature tensor is split into a 64-length sequence, which contains $B \times 256$-sized tensors. Then, the transformer encoder processes the input tensors and outputs a sequence of feature tensors whose shape is the same as that of the input. Finally, based on the encoder output and vertex queries, the transformer decoder predicts N vertices $\{v^i_{t+1}\}^N_{i=1}$, and the valid probability p^i_{t+1} of each vertex v^i_{t+1}. Vertices with high enough p^i_{t+1} are regarded as valid vertices and used for graph updates.

as $\mathcal{H} \in \mathbb{R}^{1\times L\times L}$ for afterward concatenation. With $\mathcal{S}$, $\mathcal{I}$, and $\mathcal{H}$ as input, another CNN network outputs the feature tensor $\mathcal{F}_{\mathcal{H}}$. Two feature tensors are concatenated together as the final feature tensor $\mathcal{F}$, containing all the information required by the transformer.

C. Transformer Architecture

After obtaining the deep feature tensor $\mathcal{F}$, the transformer predicts the adjacent vertices $\{v^i_{t+1}\}^M_{i=1}$ of the current vertex v_t. The feature tensor $\mathcal{F}$ is reduced to a sequence of smaller tensors and then fixed positional encoding [47], [48] is fused due to the permutation-invariant characteristics of the transformer architecture [19]. The input and output of the transformer encoder have the same length.

The decoder of the transformer takes in the output sequence of the encoder as well as a set of vertex queries $Q = \{q_i\}^N_{i=1}$ and predicts N embeddings of the adjacent vertices. Each vertex query q_i is a learned embedding and produces one predicted adjacent vertex. In fact, the transformer outputs the maximum-a-posterior (MAP) estimation of vertices at the next time step

$$\left\{ \underset{v^i_{t+1}}{\operatorname{argmax}}\, p[v^i_{t+1}|\mathcal{F}, v_t, q_i] \right\}^N_{i=1}. \tag{1}$$

Each output embedding can be decoded into a valid probability p^i_{t+1} and a 2-D vertex coordinate v^i_{t+1} by feed-forward networks (FFNs). p^i_{t+1} demonstrates the probability that v^i_{t+1} is valid and should be added into the road network graph G. Suppose we have $M(M \leq N)$ valid predicted vertices as a set $\mathcal{V} = \{v^i_{t+1}\}^M_{i=1}$, then for each valid predicted vertex v^i_{t+1}, we update the road network graph G by adding v^i_{t+1} into V and a new edge connecting v^i_{t+1} with v_t into E. The transformer architecture is visualized in Fig. 3.

D. Policy for Graph Generation During Inference Period

During the inference period, RNGDet generates the road network graph by iterations. First, we initialize the candidate initial vertex set C by finding local peaks of the road intersection segmentation map $\mathcal{I}$. Then, RNGDet pops one vertex v_t from the set C. Centering at v_t, RNGDet crops input images and predicts valid vertices at the next time step as $\mathcal{V}$. Based on the number of valid vertices $|\mathcal{V}| = M$, RNGDet takes different actions to update the road network graph: 1) $M = 0$. It means there is no road ahead, thus RNGDet should stop processing the current road and turns to work on other roads if C is not empty; 2) $M = 1$. This happens when RNGDet travels along a single road. RNGDet adds v^1_{t+1} into V and edge (v_t, v^1_{t+1}) into E and then moves to v^1_{t+1}. RNGDet keeps updating the graph in this way until intersections or broken roads are met; and 3) $M > 1$. This indicates that RNGDet encounters road intersections and needs to generate new vertices in multiple directions. RNGDet will update the graph, push all vertices in $\mathcal{V}$ to C, and pop one candidate initial vertex from C.

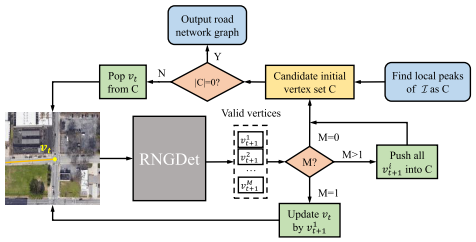


Fig. 4. Pipeline of road network graph generation by RNGDet. RNGDet iteratively generates the road network graph vertex-by-vertex. According to the number of valid vertices at each step, RNGDet takes different actions to update the graph. See zoomed-in view for details.

RNGDet keeps running the above iterations to generate the road network graph vertex-by-vertex. If and only if the candidate initial vertex set C is empty, RNGDet stops and outputs the generated road network graph $G = (V, E)$. The pipeline of the road network graph generation process of RNGDet is visualized in Fig. 4. Some example visualizations are shown in Fig. 5.

E. Training Label Calculation

Based on the ground-truth road network graph and the current location of the agent v_t, we can automatically generate the training label for RNGDet (i.e., $\mathcal{S}^*$, $\mathcal{I}^*$, $\mathcal{V}^* = \{(v^i_{t+1})^*\}^M_{i=1}$). The segmentation labels can be simply cropped from the ground-truth segmentation masks centering at v_t. For $\mathcal{V}^*$, we need to obtain coordinates of the ground-truth vertices at the next time step. To achieve that, we: 1) simplify the ground-truth road network graph by removing vertices whose degree is 2 and 2) generate the label vertices at the next time step based on the ground-truth road network graph.

The raw ground-truth graph consists of various kinds of vertices, such as endpoint vertices (degrees are 1), vertices in the middle of roads (degrees are 2), and intersection vertices (degrees are larger than 2). Among them, vertices whose degrees are 2 are not uniquely defined and could be removed without harming the road network topology. Therefore, for simplicity, we remove these vertices from the ground-truth graph. This process is visualized in Fig. 6. After the graph

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

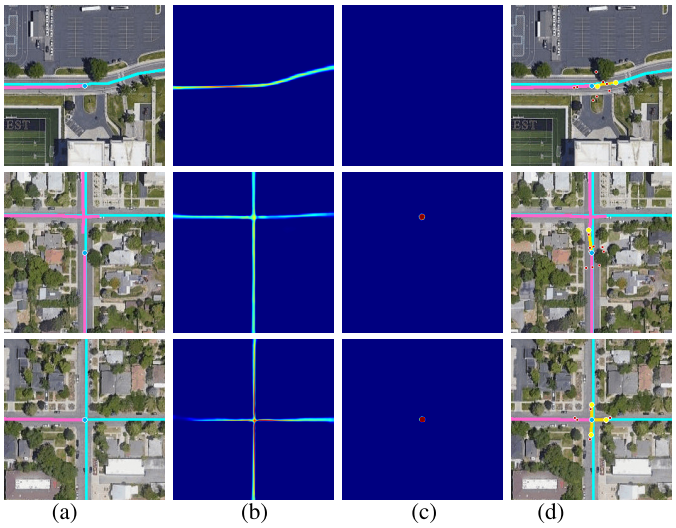


Fig. 5. Visualization of graph updating. Each row represents an example that shows how RNGDet iteratively generates the road network graph. (a) ROI of the current step t. The cyan lines are the ground-truth road network, the pink lines demonstrate history graph, and the blue node is v_t. (b) Segmentation map of road segments (i.e., $\mathcal{S}$). (c) Segmentation map of road intersections (i.e., $\mathcal{I}$). (d) Updated graph at the next time step $t+1$. Yellow nodes are valid predicted adjacent vertices v_{t+1}^i, while red nodes are invalid ones. Valid vertices will be used to update the road network graph. Orange lines are the predicted new edges connecting v_t with v_{t+1}^i. This figure is best viewed in color. See zoomed-in view for details.

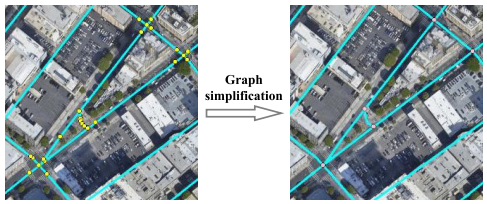


Fig. 6. Visualization of the simplification of the ground-truth road network graph. In the raw ground-truth graph, there are various kinds of vertices with different degrees (yellow nodes in the left subfigure). For training label calculation, we remove vertices whose degrees are two. After filtering, only intersection vertices (degrees larger than 2) and endpoint vertices (degrees are 1) remain (pink nodes in the right subfigure). The road connecting two adjacent vertices is defined as a road segment. The agent must finish exploring a road segment before switching to other road segments. In this way, we can guarantee the correctness of automatically generated labels. This figure is best viewed in color.

simplification, there are only endpoint vertices and intersection vertices remaining.

To facilitate the calculation of labels, we define every road connecting adjacent vertices as a road segment. The agent will be either in the road-segment mode or the intersection mode. The road-segment mode indicates that the agent is currently traveling along a road segment, and there will be only one ground-truth valid vertex at each step (i.e., $\mathcal{V}^* = \{(v_{t+1}^1)^*\}$). Three examples are visualized in Fig. 7(a)–(c). To prevent the agent from being trapped in an infinite loop, the training label should encourage the agent to move forward (i.e., $(v_{t+1}^1)^*$ should be far enough from v_t and move to the unexplored part of the current road segment). If the road ahead is straight (i.e., no turning points with large curvature within τ distance to v_t), we select the point whose distance is τ away from v_t as $(v_{t+1}^1)^*$ [Fig. 7(a)]. If there are some turning points ahead, we find the

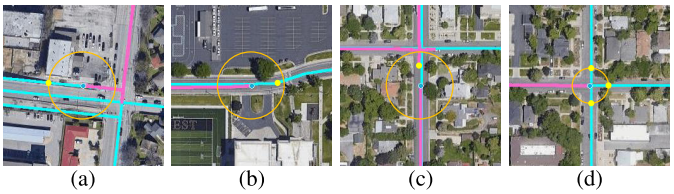


Fig. 7. Examples of training label generation. Cyan lines are the ground-truth road network, pink lines are the history road network, blue nodes represent v_t, and $(v_{t+1}^i)^*$ is denoted by yellow nodes. The radiant of the orange circle in (a)–(c) is τ, while the radiant of the orange circle in (d) is τ'. τ' is smaller than τ in our experiment. (a) Road-segment mode. The road ahead is straight. (b) Road-segment mode. The road ahead has turning points with large enough curvature. (c) Road-segment mode. The agent should connect v_t with a previously generated candidate initial vertex. (d) Intersection mode. There are three new road segments that are incident to the current intersection point. This figure is best viewed in color.

turning point that is closest to v_t as $(v_{t+1}^1)^*$ [Fig. 7(b)]. If there is some candidate initial vertices within τ distance to v_t, the closest candidate initial vertex is treated as $(v_{t+1}^1)^*$ [Fig. 7(c)]. Note that the above algorithm is only utilized to generate the training label and does not affect the graph updating process during inference.

When the agent finishes exploring the current road segment, it switches to the intersection mode and finds incident road segments. For each incident road segment, the point whose distance is τ' away from v_t is defined as the label vertex [Fig. 7(d)]. Usually, there will be multiple $(v_{t+1}^i)^*$ when RNGDet is in the intersection mode (i.e., $\mathcal{V}^* = \{(v_{t+1}^i)^*\}_{i=1}^M$ and $M > 1$).

F. Loss Functions

At each step, RNGDet predicts two segmentation maps as well as coordinates and valid probability of vertices at the next step. Thus, in our experiment, three loss functions are utilized to train RNGDet. Suppose the predicted segmentation maps are $\hat{\mathcal{S}}$ and $\hat{\mathcal{I}}$, while the ground-truth segmentation masks are $\mathcal{S}^*$ and $\mathcal{I}^*$, then we have

$$\mathcal{L}_{\text{seg}} = L(\hat{\mathcal{S}}, \mathcal{S}^*) + L(\hat{\mathcal{I}}, \mathcal{I}^*) \tag{2}$$

where L is focal loss [49]. Similarly, suppose the ground-truth vertices at the next step $t+1$ are $\{v_j^*\}_{j=1}^M$ and the predictions are $\{\hat{v}_i\}_{i=1}^N$, where $M \leq N$. They can be matched by solving a bipartite matching problem through minimizing the following function:

$$\hat{\sigma} = \underset{\sigma}{\text{argmin}} \sum_i^N \mathcal{L}_{\text{match}}(\hat{v}_i, v_\sigma^*) \tag{3}$$

where σ is the index of v^* matched with $\hat{v}_i$, and $\mathcal{L}_{\text{match}}$ calculates pair wise Euclidean distance. After the vertex matching, we have the L1 loss as

$$\mathcal{L}_{\text{coord}} = \frac{1}{N} \sum_i^N |\hat{v}_i - v_\sigma^*|. \tag{4}$$

RNGDet also predicts the valid probability $\hat{p}_i$ of each $\hat{v}_i$, and only vertices with high enough $\hat{p}_i$ will be used to update the road network graph. The ground-truth value of $\hat{p}_i$ is 1 if

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

$\hat{v}_i$ is matched with a v_σ^*, otherwise the ground-truth value of $\hat{p}_i$ is 0 (i.e., $\hat{v}_i$ does not matched with any v^*). Binary cross entropy loss is utilized to optimize $\hat{p}_i$

$$\mathcal{L}_{\text{valid}} = \text{BCELoss}(\hat{\mathcal{P}}, \mathcal{P}^*) \tag{5}$$

where $\hat{\mathcal{P}} = \{\hat{p}_i\}_{i=1}^M$ and $\mathcal{P}^* = \{p_i^*\}_{i=1}^M$. The final loss function training RNGDet is the weighted summation of the aforementioned loss functions

$$\mathcal{L} = \mathcal{L}_{\text{seg}} + \alpha\mathcal{L}_{\text{coord}} + \beta\mathcal{L}_{\text{valid}} \tag{6}$$

where α and β balance the loss function. We have $\alpha = 5$ and $\beta = 1$ in our experiments.

G. Training Data Sampling

To obtain the training dataset for RNGDet, we propose a behavior-cloning-based algorithm to explore the ground-truth road network graph to generate training samples. Suppose the agent is now at v_t, we then crop the ROI on I_A, crop $\mathcal{S}^*$ on the ground-truth road segment segmentation mask, crop $\mathcal{I}^*$ on the ground-truth road intersection segmentation mask, crop and rasterize G for $\mathcal{H}$, and calculate the ground-truth vertices as well as a valid probability at the next time step (i.e., $\mathcal{V}^*$ and $\mathcal{P}^*$) based on aforementioned approaches. Then, one training sample (ROI, $\mathcal{H}$, $\mathcal{S}^*$, $\mathcal{I}^*$, $\mathcal{V}^*$, $\mathcal{P}^*$) is obtained. If the agent is in the road-segment mode and the ground-truth vertex at the next time step $(v_{t+1}^1)^*$ is an intersection point, we use $(v_{t+1}^1)^*$ to update the graph (i.e., the agent moves to $(v_{t+1}^1)^*$); otherwise, we add $(v_{t+1}^i)^*$ with Gaussian noises Δ to update the graph (i.e., the agent moves to $(v_{t+1}^i)^* + \Delta$). The agent is driven to explore road networks by this behavior-cloning-based sampling algorithm, and the generated samples are used to train RNGDet.

IV. Experimental Results and Discussions

A. Dataset

In this article, all the experiments are conducted on the RoadTracer dataset [14]. The dataset contains 300 high-resolution aerial images (60 cm/pixel) obtained from the Google map, and the ground-truth road network graphs are from the OpenStreetMap (OSM). All the data have been converted to the image coordinate system. This dataset covers 40 cities (e.g., Los Angeles and Boston). Each aerial image has three channels and is 4096 × 4096-sized. A city may be composed of multiple aerial images.

B. Implementation

To obtain the dataset to train RNGDet, we run the proposed data sampling algorithm to explore the ground-truth road network graph of the training aerial images and generate the training samples. At each step, Gaussian noise is added for graph updating in order to make RNGDet more robust. Finally, the training set contains around 300K samples from different aerial images. We split 10K samples from the training set as the validation data.

In our experiment, we set the crop size of ROI as 256 (i.e., $L = 256$) to tradeoff between effectiveness and efficiency. When we generate the training labels, τ is set as 40 pixels and τ' is set as 20 pixels. For the transformer, the number of input vertex queries is 10 (i.e., $|Q| = N = 10$). RNGDet is trained with a learning rate of 10^{-4} and a decay rate of 10^{-5} for 50 epochs. We evaluate the performance of RNGDet on the validation set at the end of each epoch. All the experiments are conducted on four RTX-3090 GPUs.

C. Baselines

We compare our proposed RNGDet with two segmentation-based approaches and three graph-based approaches.

1) *ImprovedRoad [8] (CVPR 2019):* ImprovedRoad is one of the state-of-the-art segmentation-based approaches in the past. Orientation information is used to enhance the road segmentation, and it trains an extra refine network to fix incorrect road segmentation predictions.
2) *SPIN RoadMapper [11] (ICRA 2022):* Based on ImprovedRoad, SPIN RoadMapper proposes a graph reasoning scheme to further capture spatial information of the aerial image. ImprovedRoad and SPIN RoadMapper are trained for 120 epochs. These two approaches usually suffer from poor topology correctness.
3) *RoadTracer [14] (CVPR 2018):* RoadTracer is believed to be the first graph-based approach. It predicts the directions of the vertices at the next step as a multiclass classification problem. However, it has a fixed step size and the training label generation algorithm may produce inappropriate labels.
4) *VecRoad [17] (CVPR 2020):* VecRoad is an improved version of RoadTracer and is the state-of-the-art graph-based approach. It predicts the distribution of the vertices at the next step, which allows flexible step size. But it is still two-stage and cannot be optimized in an end-to-end manner. Moreover, it may not be able to distinguish vertices that are close to each other.
5) *Sat2Graph [16] (ECCV 2020):* Sat2Graph proposes a new graph encoding scheme and designs deep neural networks to predict the graph encoding of the input image. The predicted graph encoding can be decoded into road network graphs with satisfactory accuracy. However, it suffers from the isomorphic encoding issue which is analyzed in [16], and it has relatively inferior performance when arc roads are encountered please refer to the fourth row of Fig. 8 as an example).

D. Evaluation Metrics

In our experiments, we use three metrics pixel-precision (P-P), pixel-recall (P-R), and pixel-F1-score (P-F) for pixel-level evaluation, three metrics intersection-precision (I-P), intersection-recall (I-R), and intersection-F1-score (I-F) for intersection point evaluation, and one metric average path length similarity (APLS) [50] for topology correctness evaluation.

To calculate the pixel-level metric scores, we rasterize the ground-truth graph and the predicted graph as binary images B^* and $\hat{B}$, respectively. For a pixel in B^*, if there exists a pixel in $\hat{B}$ and the Euclidean distance between them is smaller than δ, then this pixel is treated as correctly retrieved. Similarly, if

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

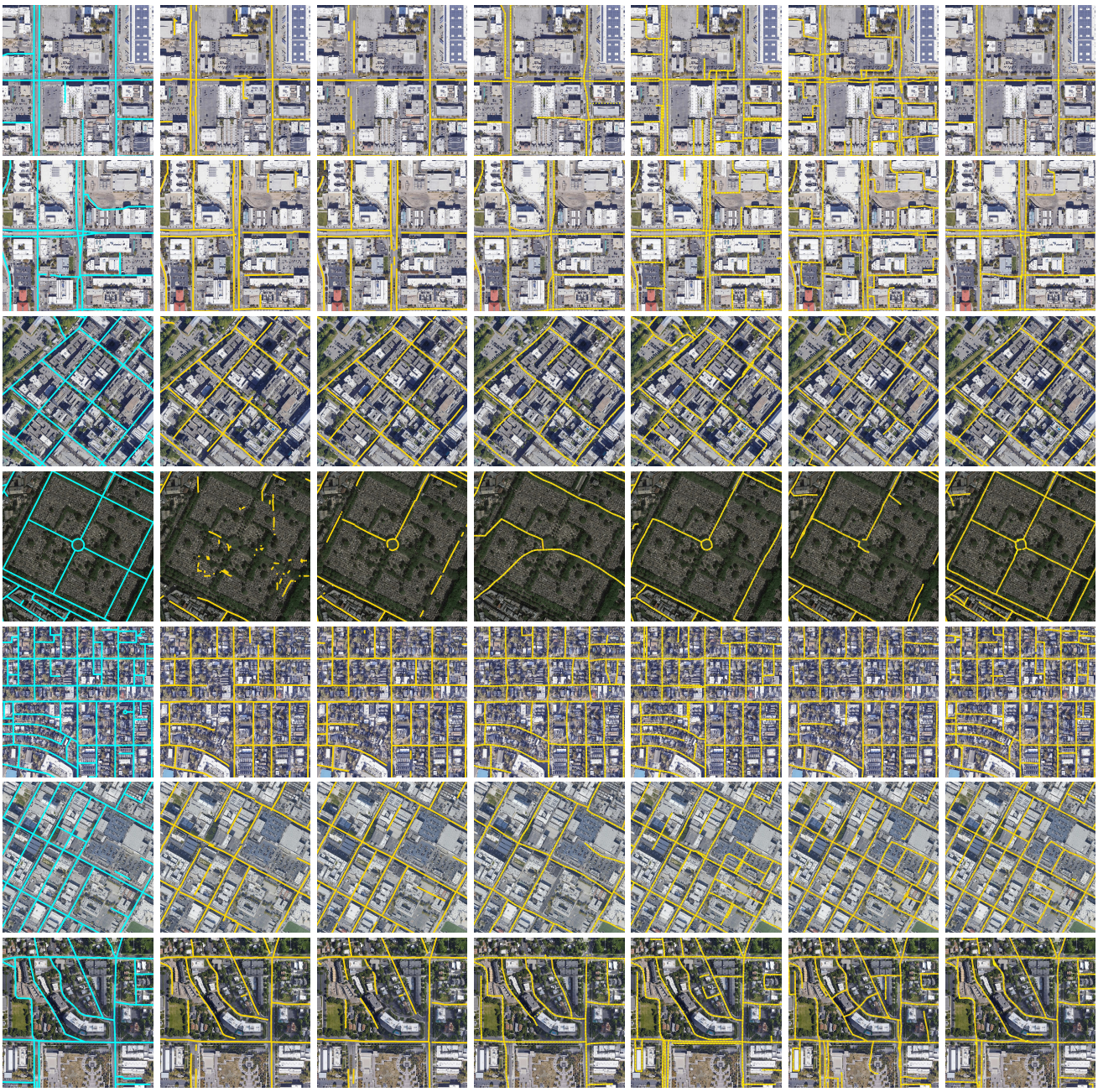


Fig. 8. Qualitative demonstrations. We visualize the road network detection results on aerial images. The size of each image is 512 × 512. (a) Ground-truth road network graph (cyan lines). (b) and (c) Road network graph predicted by segmentation-based approaches (orange lines). These two approaches have poor topology performance such as incorrect disconnections. (d)–(g) Road network graph predicted by graph-based approaches (orange lines as edges and yellow points as vertices). Compared with RoadTracer, VecRoad, and Sat2Graph, RNGDet presents more precise graph structures. For better visualization, lines are widened but they are actually of one-pixel width. This figure is best viewed in color. See zoomed-in view for details. (a) Ground truth. (b) Improved road [8]. (c) SPIN road [11]. (d) RoadTracer [14]. (e) VecRoad [17]. (f) Sat2Graph [16]. (g) RNGDet.

a pixel in $\hat{B}$ can find a pixel in B^* within δ distance, then we say this pixel is correctly detected. In this way, P-P, P-R, and P-F can be obtained by the following equations:

$$\begin{aligned} \text{P-P} &= \frac{|\{p | \|p, q\| < \delta, \exists q \in B^* \quad \forall p \in \hat{B}\}|}{|\hat{B}|} \\ \text{P-R} &= \frac{|\{p | \|p, q\| < \delta, \exists q \in \hat{B} \quad \forall p \in B^*\}|}{|B^*|} \\ \text{P-F} &= \frac{2\text{P-P} \cdot \text{P-R}}{\text{P-P} + \text{P-R}} \end{aligned} \tag{7}$$

where $\|\cdot\|$ calculates the Euclidean distance and $|\cdot|$ is the cardinality of a set. Threshold δ measures the level of error tolerance, and we show the metric scores with δ as 2, 5, and 10 pixels for more comprehensive evaluation.

I-P, I-R, and I-F are calculated in a similar way as the aforementioned pixels-level metrics. The only difference is that instead of rasterizing the whole graph into binary images B^* and $\hat{B}$, these three metrics only care about intersection points (i.e., rasterized binary images only contain intersection points). These three metrics evaluate the ability of approaches

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I

QUANTITATIVE COMPARISON RESULTS. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD FONT. FOR ALL THE METRICS, LARGER VALUES INDICATE BETTER PERFORMANCE

Approaches	P-P ↑			P-R ↑			P-F ↑			I-P ↑			I-R ↑			I-F ↑			APLS ↑
	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	
ImprovedRoad [8]	68.22	75.70	78.72	47.50	54.09	57.80	56.01	63.10	66.66	29.00	40.62	43.10	23.25	32.60	34.52	25.81	36.17	38.34	41.71
SPIN RoadMapper [11]	**78.53**	**85.82**	**89.02**	54.02	60.28	63.71	64.01	70.82	74.27	**45.27**	59.66	62.08	28.87	37.56	38.93	33.67	35.26	46.10	47.85
RoadTracer [14]	57.49	68.26	74.53	35.09	41.81	46.28	43.58	51.86	57.10	22.79	55.50	**78.93**	13.18	32.11	44.16	16.59	16.70	40.68	56.63
VecRoad [17]	60.87	69.33	73.97	64.91	74.00	78.88	62.83	71.59	76.35	37.49	63.87	68.77	37.51	63.70	68.43	37.50	63.78	68.60	65.69
Sat2Graph [16]	57.87	65.33	69.43	63.02	71.33	76.15	60.33	68.20	72.64	29.61	57.02	60.33	34.20	65.90	69.64	31.74	61.14	64.65	63.21
RNGDet (ResNet-34)	58.33	69.07	73.81	63.80	**75.15**	79.43	60.94	71.98	76.52	32.83	57.02	69.22	34.19	55.96	72.16	33.50	56.49	70.66	55.64
RNGDet (ResNet-50)	62.11	69.54	74.39	66.07	74.92	80.05	64.03	72.12	77.13	36.51	63.97	75.20	39.98	65.48	62.84	38.17	64.72	68.47	63.76
RNGDet (ResNet-101)	65.63	72.31	77.08	**66.42**	75.08	**82.13**	**66.02**	**73.67**	**79.52**	42.37	**65.30**	72.18	**40.40**	**66.50**	**73.23**	**41.36**	**65.89**	**72.70**	**67.88**

to detect road intersections. We also show these metric scores in our experimental results with different δ.

APLS measures the similarity of the ground-truth graph G^* and the predicted graph $\hat{G}$. It samples multiple vertex pairs on both graphs and compares the difference between the shortest distance of vertex pairs. APLS is calculated by the following equation:

$$\text{APLS} = 1 - \frac{1}{N_s}\sum^{N_s} \min\left(1, \frac{|L(a,b) - L(a',b')|}{L(a,b)}\right) \quad (8)$$

where (a, b) demonstrates a vertex pair sampled from G^* and (a', b') is the corresponding vertex pair sampled from $\hat{G}$. The number of sampled vertex pairs is denoted by N_s, and $L(\cdot, \cdot)$ calculates the length of the shortest path between two vertices. Higher APLS indicates better graph similarity and topology correctness.

E. Comparative Results

RNGDet is compared with five baseline approaches, including two segmentation-based baselines and three graph-based baselines. The quantitative comparison results are shown in Table I. In Table I, besides baseline approaches, we also evaluate RNGDet with different backbones (i.e., ResNet-34, ResNet-50, and ResNet-101) for more fair and comprehensive comparison. Qualitative visualizations are provided in Fig. 8.

For segmentation results, we first predict the segmentation map and then binarize it by thresholding. Finally, we run skeletonization algorithms to extract the graph of predicted segmentation maps. From the comparison results, we find that these approaches have good pixel-level results since they directly optimize pixel-level segmentation. However, since they cannot fully utilize spatial and geometric information, they have poor performance on topology correctness. Thus, segmentation-based approaches have relatively inferior intersection-level and topology-level scores. Therefore, segmentation-based approaches are not sufficient for our road network graph detection task.

Graph-based approaches directly optimize the graph, thus they present much better results from the topology perspective. RoadTracer has a fixed step length, which makes it unable to handle some scenarios very well, especially when the agent is near road intersections. VecRoad is more powerful due to the use of Res2Net [51] backbone network and the flexible step size. However, VecRoad predicts vertices by finding local peaks on heatmaps, so when vertices are very close to each other, it may not be able to correctly detect them. Therefore, it may fail to correctly detect some precise graph structures. Sat2Graph presents good results on straight road detection, but it cannot handle curve roads very well, which degrades its final performance (refer to the fourth row of Fig. 8 as an example). Our proposed RNGDet can directly output the coordinates of vertices, and therefore RNGDet can handle more complicated situations and has a more powerful performance. RNGDet with ResNet-101 backbone has the best performance during evaluation, gaining 2%–5% improvement on almost all metrics scores compared with past state-of-the-art approaches. RNGDet with ResNet-50 backbone has a little bit inferior performance, but it still outperforms VecRoad except APLS. RNGDet with ResNet-34 backbone presents unsatisfactory results, which may be caused by the shallow backbone network and FPN segmentation heads.

F. Ablation Studies

In this section, we study the significance of some components of our network design, including the road segment segmentation $\mathcal{S}$, road intersection segmentation $\mathcal{I}$, and the transformer structure of RNGDet. All ablation studies are conducted on RNGDet with ResNet-101 backbone. The quantitative results of our ablation studies are shown in Table II.

First, we completely remove the segmentation heads from RNGDet. From the evaluation results, we notice that the performance of RNGDet drops a lot because the segmentation heads can provide strong supervision to assist the CNN backbone to extract features of the input. Then, we evaluate RNGDet by only removing the road segment segmentation $\mathcal{S}$. All metrics scores degrade, especially pixel-level metrics scores. After this, we remove the road intersection segmentation $\mathcal{I}$ only, and the intersection-level metrics scores and APLS are severely harmed. Because $\mathcal{I}$ is critical to make RNGDet aware of road intersections, removing $\mathcal{I}$ will lead to incorrect connections near road intersections and make the topology correctness much worse. In this way, the necessity of the segmentation maps including $\mathcal{S}$ and $\mathcal{I}$ is verified.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

TABLE II

QUANTITATIVE RESULTS FOR THE ABLATION STUDY. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD FONT. FOR ALL THE METRICS, LARGER VALUES INDICATE BETTER PERFORMANCE. WE ASSESS THE ROAD SEGMENT SEGMENTATION ($\mathcal{S}$), THE ROAD INTERSECTION SEGMENTATION ($\mathcal{I}$) AND THE TRANSFORMER STRUCTURE (T)

			P-P ↑			P-R ↑			P-F ↑			I-P ↑			I-R ↑			I-F ↑			APLS ↑
$\mathcal{S}$	$\mathcal{I}$	T	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	2.0	5.0	10.0	
		✓	37.63	56.80	61.79	55.24	60.12	64.77	44.77	58.41	63.24	25.40	47.03	69.62	20.70	40.93	53.26	22.81	43.77	60.35	50.32
	✓	✓	40.09	55.83	65.97	57.41	63.28	66.21	47.21	59.32	66.09	36.95	56.38	73.72	33.39	58.64	64.30	35.08	57.49	68.69	61.22
✓		✓	59.80	65.34	72.79	62.19	73.83	79.02	60.97	69.33	75.78	32.14	50.86	68.44	32.62	55.04	62.47	32.38	52.87	65.32	59.90
✓	✓		56.04	69.91	74.52	62.88	71.64	74.56	59.26	70.76	74.54	31.65	57.21	63.90	32.10	59.33	62.42	31.87	58.25	63.15	60.79
✓	✓	✓	**65.63**	**72.31**	**77.08**	**66.42**	**75.08**	**82.13**	**66.02**	**73.67**	**79.52**	**42.37**	**65.30**	**72.18**	**40.40**	**66.50**	**73.23**	**41.36**	**65.89**	**72.70**	**67.88**

TABLE III

QUANTITATIVE RESULTS OF RNGDET WITH DIFFERENT NUMBERS OF QUERIES ($|Q|$)

	P-F ↑	I-F ↑	APLS ↑
RNGDet with 5 queries	73.10	61.74	62.78
RNGDet with 10 queries	73.67	65.89	67.88
RNGDet with 20 queries	73.90	64.27	65.31

Finally, to examine the importance of the transformer structure in RNGDet, we replace the transformer with a modified mask-RCNN [52]. In our task, similar to VecRoad [17], mask-RCNN first predicts the heatmap of vertices at the next time step and then extracts the coordinate of vertices by handcrafted or heuristic postprocessing. Thus, RNGDet with mask-RCNN cannot be optimized in an end-to-end way either, which affects its final performance. From the experimental results, RNGDet with mask-RCNN presents inferior results compared with the original RNGDet. Therefore, the importance of the transformer structure is proved.

G. Number of Vertex Queries

Under normal circumstances, the number of queries $|Q|$ should be obviously larger than the maximum number of vertices at the next time step. Usually, the road intersections have at most five roads incident with each other. Thus, we try to train RNGDet with 5, 10, and 20 queries and observe the obtained performance. We use RNGDet with ResNet-101 for the experiments. During the experiments, we use metrics P-F ($\delta = 5$), I-F ($\delta = 5$), and APLS to evaluate models.

Based on the results shown in Table III, RNGDet with ten input queries presents the best performance, thus the number of queries is set to 10.

H. Failure Cases

Although RNGDet presents superiority against past approaches, it still cannot handle some very complicated cases, such as occluded overpasses, very well yet. Moreover, RNGDet also suffers from the drifting issue of imitation learning (i.e., when the agent is away from the right track, it may not be able to get back to the correct state) similar to other graph-based approaches, even though it can relieve this problem to some extent. Some example visualizations of failure cases of RNGDet are shown in Fig. 9. These cases could be handled in the future with more powerful backbone networks or training strategies.

Fig. 9. Qualitative demonstrations of the failure cases. (a) Ground truth (cyan lines). (b) Result of VecRoad (orange lines for edges and yellow points for vertices). (c) Result of RNGDet (orange lines for edges and yellow points for vertices). The first and second rows show complicated road intersections and overpasses with occlusion. The last row is an example containing severe tree occlusion. RNGDet at this stage cannot handle these cases very well yet. This problem could be relieved in the future by using more powerful backbone networks or training strategies for RNGDet. The figure is best viewed in color. See zoomed-in view for details.

V. CONCLUSION

We proposed here RNGDet, a novel iterative approach to automatically detect the road network graph from aerial images. Taken as input an aerial image, RNGDet could directly output the road network graph with vertices and edges. First, RNGDet predicted a set of candidate initial vertices and then iteratively generated the road network graph

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

vertex-by-vertex starting from each candidate initial vertex. Due to the use of transformers and deep queries, RNGDet could handle complicated intersection points with an arbitrary number of incident road segments. RNGDet was evaluated on a publicly available dataset and presented the state-of-the-art performance in terms of all evaluation metrics, including pixel-level metrics, intersection-level metrics, and topology-level metric APLS. The experimental results demonstrated the superiority of our work. In the future, we would like to further improve RNGDet by using more powerful backbone networks and training strategies. Besides, we will adapt RNGDet to other graph detection tasks, such as road laneline detection and road lane centerline detection.

REFERENCES

[1] Y.-Y. Chiang and C. A. Knoblock, "Extracting road vector data from raster maps," in *Proc. Int. Workshop Graph. Recognit.* Berlin, Germany: Springer, 2009, pp. 93–105.

[2] V. Mnih and G. E. Hinton, "Learning to detect roads in high-resolution aerial images," in *Proc. Eur. Conf. Comput. Vis.* Berlin, Germany: Springer, 2010, pp. 210–223.

[3] (2019). N. O. Department of Information Technology & Telecommunications (DoITT). *NYC-Planimetrics Database*. [Online]. Available: https://github.com/CityOfNewYork/nyc-planimetrics

[4] X. Hu, Y. Li, J. Shan, J. Zhang, and Y. Zhang, "Road centerline extraction in complex urban scenes from LiDAR data based on multiple features," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 11, pp. 7448–7456, Nov. 2014.

[5] W. Shi, Z. Miao, Q. Wang, and H. Zhang, "Spectral–spatial classification and shape features for urban road centerline extraction," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 4, pp. 788–792, Apr. 2013.

[6] C. Unsalan and B. Sirmacek, "Road network detection using probabilistic and graph theoretical methods," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 11, pp. 4441–4453, Nov. 2012.

[7] G. Cheng, F. Zhu, S. Xiang, and C. Pan, "Road centerline extraction via semisupervised segmentation and multidirection nonmaximum suppression," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 4, pp. 545–549, Feb. 2016.

[8] A. Batra, S. Singh, G. Pang, S. Basu, C. V. Jawahar, and M. Paluri, "Improved road connectivity by joint learning of orientation and segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10385–10393.

[9] G. Mattyus, W. Luo, and R. Urtasun, "DeepRoadMapper: Extracting road topology from aerial images," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 3438–3446.

[10] A. V. Etten, "City-scale road extraction from satellite imagery v2: Road speeds and travel times," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2020, pp. 1786–1795.

[11] W. G. C. Bandara, J. M. J. Valanarasu, and V. M. Patel, "SPIN road mapper: Extracting roads from aerial images via spatial and interaction space graph reasoning for autonomous driving," 2021, *arXiv:2109.07701*.

[12] G. Cheng, Y. Wang, S. Xu, H. Wang, S. Xiang, and C. Pan, "Automatic road detection and centerline extraction via cascaded end-to-end convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 6, pp. 3322–3337, Jun. 2017.

[13] G. Zhou, W. Chen, Q. Gui, X. Li, and L. Wang, "Split depth-wise separable graph-convolution network for road extraction in complex environments from high-resolution remote-sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2021.

[14] F. Bastani *et al.*, "RoadTracer: Automatic extraction of road networks from aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4720–4728.

[15] Z. Li, J. D. Wegner, and A. Lucchi, "Topological map extraction from overhead images," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2019, pp. 1715–1724.

[16] S. He *et al.*, "Sat2Graph: Road graph extraction through graph-tensor encoding," in *Computer Vision*. Glasgow, U.K.: Springer, Aug. 2020, pp. 51–67.

[17] Y.-Q. Tan, S.-H. Gao, X.-Y. Li, M.-M. Cheng, and B. Ren, "VecRoad: Point-based iterative graph exploration for road graphs extraction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 8910–8918.

[18] D. Belli and T. Kipf, "Image-conditioned graph generation for road network extraction," *Proc. NIPS workshop Graph Represent. Learn.*, 2019, pp. 1–9.

[19] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 213–229.

[20] S. Ganguli, P. Garzon, and N. Glaser, "GeoGAN: A conditional GAN with reconstruction and style loss to generate standard layer of maps from satellite images," 2019, *arXiv:1902.05611*.

[21] R. He, X. Li, G. Chen, G. Chen, and Y. Liu, "Generative adversarial network-based semi-supervised learning for real-time risk warning of process industries," *Expert Syst. Appl.*, vol. 150, Jul. 2020, Art. no. 113244.

[22] Y. Kang, S. Gao, and R. E. Roth, "Transferring multiscale map styles using generative adversarial networks," *Int. J. Cartography*, vol. 5, nos. 2–3, pp. 115–141, May 2019.

[23] S. Wenzel and D. Bulatov, "Simultaneous chain-forming and generalization of road networks," *Photogramm. Eng. Remote Sens.*, vol. 85, no. 1, pp. 19–28, Jan. 2019.

[24] D. Bulatov, S. Wenzel, G. Häufel, and J. Meidow, "Chain-wise generalization of road networks using model selection," *ISPRS Ann. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 4, p. 59, May 2017.

[25] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, "Sequence level training with recurrent neural networks," 2015, *arXiv:1511.06732*.

[26] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 801–818.

[27] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2117–2125.

[28] J. Wang *et al.*, "Deep high-resolution representation learning for visual recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3349–3364, Oct. 2021.

[29] J. Liang, N. Homayounfar, W.-C. Ma, S. Wang, and R. Urtasun, "Convolutional recurrent network for road boundary extraction," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2019, pp. 9512–9521.

[30] Z. Xu, Y. Sun, and M. Liu, "Topo-boundary: A benchmark dataset on topological road-boundary detection using aerial images for autonomous driving," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7248–7255, Oct. 2021.

[31] Z. Xu *et al.*, "CsBoundary: City-scale road-boundary detection in aerial images for high-definition maps," 2021, *arXiv:2111.06020*.

[32] N. Homayounfar, W.-C. Ma, S. K. Lakshmikanth, and R. Urtasun, "Hierarchical recurrent attention networks for structured online maps," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3417–3426.

[33] N. Homayounfar, W.-C. Ma, J. Liang, X. Wu, J. Fan, and R. Urtasun, "DAGMapper: Learning to map by discovering lane topology," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2019, pp. 2911–2920.

[34] Q. Li, Y. Wang, Y. Wang, and H. Zhao, "HDMapNet: An online HD map construction and evaluation framework," 2021, *arXiv:2107.06307*.

[35] X. Yang, X. Li, Y. Ye, R. Y. K. Lau, X. Zhang, and X. Huang, "Road detection and centerline extraction via deep recurrent convolutional neural network U-Net," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7209–7220, Sep. 2019.

[36] S. He and H. Balakrishnan, "Lane-level street map extraction from aerial imagery," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2022, pp. 2080–2089.

[37] Y. Zhou, Y. Takeda, M. Tomizuka, and W. Zhan, "Automatic construction of lane-level HD maps for urban scenes," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 6649–6656.

[38] Z. Xu *et al.*, "ICurb: Imitation learning-based detection of road curbs using aerial images for autonomous driving," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1097–1104, Feb. 2021.

[39] Z. Xu, Y. Sun, L. Wang, and M. Liu, "CP-loss: Connectivity-preserving loss for road curb detection in autonomous driving with aerial images," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 1117–1123.

[40] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2011, pp. 627–635.

[41] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.

[42] Y. Xu, W. Xu, D. Cheung, and Z. Tu, "Line segment detection using transformers without edges," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 4257–4266.
[43] Y. B. Can, A. Liniger, D. P. Paudel, and L. Van Gool, "Structured bird's-eye-view traffic scene understanding from onboard images," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 15661–15670.
[44] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, "An algorithmic perspective on imitation learning," 2018, *arXiv:1811.06711*.
[45] S. Ross and D. Bagnell, "Efficient reductions for imitation learning," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 661–668.
[46] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.
[47] I. Bello, B. Zoph, A. Vaswani, J. Shlens, and Q. V. Le, "Attention augmented convolutional networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2019, pp. 3286–3295.
[48] N. Parmar *et al.*, "Image transformer," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4055–4064.
[49] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.
[50] A. Van Etten, D. Lindenbaum, and T. M. Bacastow, "SpaceNet: A remote sensing dataset and challenge series," 2018, *arXiv:1807.01232*.
[51] S.-H. Gao, M.-M. Cheng, K. Zhao, X.-Y. Zhang, M.-H. Yang, and P. Torr, "Res2Net: A new multi-scale backbone architecture," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 2, pp. 652–662, Feb. 2021.
[52] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 2961–2969.

Zhenhua Xu (Graduate Student Member, IEEE) received the bachelor's degree from the Harbin Institute of Technology, Harbin, China, in 2018. He is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong, under the supervision of Prof. Ming Liu and Prof. Huamin Qu.

His research interests include HD map automatic annotation, line-shaped object detection, imitation learning, and autonomous driving.

Yuxuan Liu (Graduate Student Member, IEEE) received the bachelor's degree in mechatronic from Zhejiang University, Zhejiang, China, in 2019. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

His research interests include autonomous driving, deep learning, robotics, visual 3-D object detection, and visual depth prediction.

Lu Gan (Student Member, IEEE) received the bachelor's and master's degrees from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2012 and 2015, respectively. She is currently pursuing the Ph.D. degree with The Hong Kong University of Science and Technology, Hong Kong.

She worked with Nanyang Technological University, Singapore, as a Research Associate. Her research interests include autonomous driving, deep learning, and uncertainty-aware motion prediction and planning.

Yuxiang Sun (Member, IEEE) received the bachelor's degree from the Hefei University of Technology, Hefei, China, in 2009, the master's degree from the University of Science and Technology of China, Hefei, in 2012, and the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2017.

He is currently a Research Assistant Professor at the Department of Mechanical Engineering, The Hong Kong Polytechnic University, Hong Kong. Prior to that, he was a Research Associate with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong. His research interests include autonomous driving, deep learning, robotics and autonomous systems, and semantic scene understanding.

Dr. Sun servers as an Associate Editor for the IEEE ROBOTICS AND AUTOMATION LETTERS, the IEEE International Conference on Robotics and Automation, and the IEEE/RSJ International Conference on Intelligent Robots and Systems.

Xinyu Wu (Senior Member, IEEE) received the B.E. and M.E. degrees from the Department of Automation, University of Science and Technology of China, Hefei, China, in 2001 and 2004, respectively, and the Ph.D. degree from the Chinese University of Hong Kong, Hong Kong, in 2008.

He is currently a Professor with the Shenzhen Institutes of Advanced Technology, Shenzhen, China, and also the Director of the Center for Intelligent Bionic. He has authored or coauthored over 180 articles and two monographs. His research interests include computer vision, robotics, and intelligent systems.

Ming Liu (Senior Member, IEEE) received the B.A. degree from Tongji University, Shanghai, China, in 2005, and the Ph.D. degree from ETH Zürich, Zürich, Switzerland, in 2013.

He was with Erlangen-Nünberg University, Erlangen, Germany, and the Fraunhofer Institute IISB, Germany, as a Visiting Scholar. He is currently an Associate Professor with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong. He has been involved in several NSF projects and National 863-Hi-Tech-Plan projects in China. He is a PI of 20+ projects including projects funded by Research Grants Council (RGC), National Natural Science Foundation of China (NSFC), Innovation and Technology Commission (ITC), and Shenzhen Science and Technology Innovation Committee (SZSTI). His research interests include dynamic environment modeling, 3-D mapping, machine learning, and visual control.

Dr. Liu was the General Chair of ICVS-2017, the Program Chair of IEEE-RCAR 2016, and the Program Chair of the International Robotic Alliance Conference in 2017.

Lujia Wang (Member, IEEE) received the Ph.D. degree from the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong, in 2015.

She was a Research Fellow with the School of Electrical Electronic Engineering, Nanyang Technological University, Singapore, from 2015 to 2016. She was an Associate Professor with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China, from 2016 to 2021. Her research interests include cloud robotics, lifelong federated robotic learning, resource/task allocation for robotic systems, and applications for autonomous driving.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:56:57 UTC from IEEE Xplore. Restrictions apply.